Milestone-1 Report

NLP Foundation: TF-IDF Vectorizer for answering Multiple-Choice Questions

Naini Diwan | Roll No. 23f3001480

Smart MCQ Solver Challenge (Kaggle Competition)

1. Introduction

A baseline system was built to solve multiple-choice questions (MCQs) using classical NLP techniques, developed for the Smart MCQ Solver, a Private Challenge in Kaggle, hosted by IIT Madras. Each record in the dataset consists of a question prompt and five candidate answers, with the task being to identify the correct option. The approach taken here treats answer selection as a semantic similarity problem: rather than training a classifier, the prompt and each of its five options are converted into numerical vectors. The option whose vector is most similar to the prompt's vector as per cosine similarity, is selected as the predicted answer. The study explores the efficacy of frequency-based text vectorization and semantic distance metrics in selecting the most accurate answer from a set of predefined options. This is a reasonable first baseline because it requires no labelled training beyond fitting a vocabulary, and it gives a lower bound against which more sophisticated models (transformers, fine-tuned classifiers) can be compared.

2. Data Preprocessing

After an *Exploratory Data Analysis* was performed, the following was observed:

- The training dataset encompasses 2000 records and features no missing values.
- There were 242 duplicated question prompts within the dataset.

To maintain data integrity and prevent data leakage, only the first occurrence of each duplicate prompt was retained, yielding a finalized dataset of 1758 unique rows.

Standardization of the text data was required prior to feature extraction, because it is a bag-of-words style model, where every distinct string is a separate feature. The following preprocessing pipeline was applied:

Case normalization: All text strings across the prompt and option columns were converted to lowercase to ensure uniformity (in the previous versions only prompts were being converted to lower-case). This reduces vocabulary sparsity by treating words such as "Heidegger" and "heidegger" as the same token.

Boilerplate Removal: Redundant instructional phrases, such as "pick the best possible answer:" and "identify the correct statement:", were stripped from the prompts using regex (regular expressions).

Label Encoding: The categorical target variables representing the correct answers were mapped to numerical arrays using sklearn's label encoder.

Data Splitting: The pre-processed dataset was partitioned into an 80% training set containing 1406 records and a 20% validation set. A fixed random seed was utilized to guarantee reproducibility.

Note: The latter three were preprocessing steps are introduced to improve accuracy as compared to previous versions of the jupyter notebook. *Punctuation handling* is not performed this time, because some scientific information from questions/options was also being lost because removing all the special characters.

3. Methodology

The predictive modelling approach relies on term frequency-inverse document frequency (TF-IDF) vectorization along with cosine-similarity to evaluate the semantic alignment between prompts and options. A fixed random seed (42) was set across random, numpy, and the PYTHONHASHSEED environment variable for reproducibility.

To avoid data leakage, the vectorizer was fit only on the training split. The fitting corpus combined every prompt in the training set with every value from all five option columns (A–E), giving the vectorizer visibility into the full range of vocabulary it would need at inference time.

For each row, the prompt was transformed into a TF-IDF vector, and each of the five options was transformed the same way. Cosine similarity was computed between the prompt vector and each of the five option vectors, producing five similarity scores per question. These scores serve as a proxy for the model's confidence in each option (standing in for the logits a trained classifier outputs).

Why TF-IDF over CountVectorizer?

A raw count-based vectorizer treats every word as equally informative, which lets frequent but low-information words (e.g. "the," "is," "what") dominate the vector. TF-IDF downweighs terms that appear several times in the corpus and upweights terms that are distinctive to a smaller subset.

A maximum document frequency threshold (max_df) of 0.95 was enforced, which ignores vocabulary appearing in over 95% of the text. Then the vectorizer was fitted on a combined corpus of all training prompts and options, ensuring that every text element shares the exact same vector space.

Why cosine similarity over Euclidean distance?

Cosine similarity measures the angle between two vectors, not their magnitude. Synonymous words can produce vectors of very different lengths simply because they have different number of letters. Euclidean distance is sensitive to this length difference; whereas cosine similarity measures the semantic direction of the text vectors independently of their overall length.

4. Evaluation Metrics

Model performance was assessed using raw prediction logits mapped against the true encoded labels. The framework outputs 3 distinct metrics, each capturing a different notion of correctness.

Accuracy- The proportion of questions where the top-ranked option (highest cosine similarity) matches the true answer. This is the strictest metric as it only accounts for top-1 match.

F1 score (macro-average)- *It* computes precision and recall for each answer class (A through E) independently, then averages them without weighting by class frequency. A model could get high accuracy purely by favouring one or two highly occurring answer letters. But macro F1 penalizes such imbalance and gives a fairer picture of whether the model performs consistently across all options.

MAP@3 (Mean Average Precision at 3)- Instead of requiring the correct answer to be ranked first, it checks whether the correct answer appears anywhere in the top 3 ranked predictions, and rewards it more if it appears earlier. Concretely, for each question, if the true answer is found at rank 1, 2 or 3, the score contribution is 1, 1/2 or 1/3 respectively; otherwise the contribution is 0. These per-question scores are averaged across entire validation set to produce MAP@3, which is bounded between 0 (no correct answers ever in the top 3) and 1 (correct answer always ranked first).

5. Results

The model was evaluated on the validation split (20% of training data), yielding the following scores:

METRIC	SCORE
Accuracy	0.176
F1 Score (macro)	0.171
MAP@3	0.297

Whereas the last version of the jupyter notebook gave these results:

METRIC	SCORE
Accuracy	0.127
F1 Score (macro)	0.126
MAP@3	0.258

6. Discussion

The metrics were improved if compared with the previous methodologies, yet, with 5 options per question, random guessing would produce roughly 0.20 accuracy. MAP@3 is closest to the reasoning ability of the model, because it credit the top 3 options in sequence, over which the model has confidence. Still this is an underperformance of the model, pointing to a mismatch between the technique and the task. TF-IDF and cosine similarity detect when a prompt and an option share the same words (lexical overlap). But looking at the sample data, the correct answer is the option that is *semantically* closest, even though it shares very little vocabulary with the question itself.

Bag-of-words vectorization is very weak at such scenarios, since both produce similar TF-IDF vectors. So wrong options could seem to model mathematically closer to the correct answer, yet be semantically opposite (e.g. "humans exist within time" vs. "humans do not exist within time"). This explains why accuracy and F1 score underperform random guessing.

6. Next Steps

This baseline confirms that lexical-overlap methods like TF-IDF coupled with cosine similarity are insufficient for tasks where correctness hinges on meaning more than word overlap. The exercise was still valuable, it established a reproducible foundation, over which more capable approaches can be built. Next steps include applying dense transformers instead of TF-IDF, which encode meaning and handle negation and paraphrase better.